

Inara Technologies

Solutions for Digital Age

Basic Training Course Content

1-Month Basic Training for Software Developers

Prerequisite to the Advanced Software Development Training Course

June 23, 2026

Table of Contents

Table of Contents	2
Program Overview	3
Week 1: Linux & Shell Foundations.....	4
Week 2: Git & Networking Foundations.....	6
Week 3: SSH & Docker Foundations.....	8
Week 4: Application Environment & Capstone	9
What Comes Next: Advanced Software Development Training	11

Program Overview

This 1-month Basic Training course prepares Software Developer trainees with the supporting environment and tooling skills assumed by the Advanced Software Development Training course (Frontend Developer, Backend Developer, and UI/UX Designer tracks). Trainees are assumed to already have basic coding ability; this course does not teach programming fundamentals from scratch. Instead it covers the Linux/shell, networking, SSH, Docker, and Git skills that the Advanced course's content builds directly on top of, condensed from the original two-month Basic Training course.

The original course did not include Git, which the Advanced course's Web Foundations and Backend Foundations sections explicitly assume ("Git essentials: branches, commits, pull requests"). A new 3-day Git module has been added to close this gap. SQL and relational database fundamentals are intentionally left to the Advanced course's Data Layer section, where they are taught in depth as part of the Backend Developer track.

Each day card below is tagged to show whether its content is condensed from the original Basic Training course or newly added for this software developer path:

From original course	Condensed directly from the original two-month Basic Training course
New: Git module	Newly added — not in the original course, but assumed by the Advanced course

Course	Duration	Leads Into
Basic Training (this course)	4 weeks (20 days)	Advanced Software Development Training
Advanced Software Development Training	Frontend / Backend / UI-UX tracks	Capstone project per track

Week 1: Linux & Shell Foundations

Core command-line, permissions, and scripting skills every developer will use daily, regardless of track.

Day 1 Shell Basics & File System Hierarchy

- Navigating the command line: pwd, ls, cd, mkdir, touch, cp, mv, rm
- Command structure, arguments and options
- Absolute vs relative paths; standard directory layout
- Input/output redirection and pipes (>, >>, |)

Practical Task: Guided shell exercises: create a project folder structure, move/rename files, chain commands with pipes

Source: Original course: Shell basics, File system hierarchy

Day 2 Permissions, Users & Sudo

- Read, write, execute model; octal and symbolic notation (chmod)
- Ownership and group membership (chown, chgrp)
- The sudo model and switching users safely

Practical Task: Fix a set of broken permission scenarios on a practice project directory

Source: Original course: Permissions deep dive, Users/groups and sudo

Day 3 Text Processing with grep, sed, awk

- Searching text with grep
- Stream editing with sed (find and replace)
- Field-based processing with awk
- Combining tools through pipes for quick log/code searches

Practical Task: Extract and summarise data from a sample application log file

Source: Original course: Text processing with grep, sed, awk and pipes

Day 4 Software & Package Management, Processes

- Package managers: installing, updating, removing packages
- Viewing and managing running processes (ps, top, kill)
- Foreground/background jobs

Practical Task: Install a runtime/toolchain package, verify the version, and manage a running process

Source: Original course: Software and package management, Process management

Day 5
Shell Scripting
Intro

- Variables and quoting
- Conditionals and loops
- Exit codes
- Writing a simple reusable script

Practical Task: Write a small shell script that automates a repetitive setup task (e.g. project scaffolding)

Source: Original course: Shell scripting intro

Week 2: Git & Networking Foundations

Version control workflow (new module) plus the networking and HTTP fundamentals that the Frontend and Backend tracks both assume on day one.

Day 6 Git Fundamentals I: Local Workflow

- What version control is and why it matters
- git init, add, commit, status, log
- Writing meaningful commit messages
- Viewing history and diffs

Practical Task: Initialise a repo, make a series of commits tracking incremental changes to a small project

Source: New: Git module (added — assumed by Advanced course, not in original)

Day 7 Git Fundamentals II: Branching & Merging

- Creating and switching branches
- Merging branches; understanding merge conflicts
- Resolving a merge conflict by hand

Practical Task: Create a feature branch, introduce a conflict, and resolve it

Source: New: Git module (added)

Day 8 Git Fundamentals III: Remote Workflow & Collaboration

- Remotes: clone, push, pull, fetch
- Pull requests and the code review habit
- Common team workflows (feature branches, trunk-based basics)
- .gitignore and keeping repos clean

Practical Task: Push a project to a shared remote, open a pull request, and complete a peer review cycle

Source: New: Git module (added) / referenced in Advanced course's Web Foundations and Backend Foundations

Day 9 Networking Fundamentals: OSI/TCP-IP & Addressing

- The OSI and TCP/IP layered models (high level)
- IP addressing basics and the idea of ports
- Diagnostic tools: ping, traceroute, ss/netstat

Practical Task: Use ping, traceroute, and ss to diagnose a simple connectivity scenario

Source: Original course: OSI/TCP-IP models, IP addressing, Diagnostic tools (condensed)

Day 10
DNS, HTTP, HTTPS
& TLS

- The DNS resolution flow; querying with dig/nslookup
- The HTTP request/response cycle and common status codes
- TLS and SSL certificates at a high level

Practical Task: Use curl to inspect HTTP requests/responses and a browser's dev tools Network tab to trace a page load

Source: Original course: DNS, HTTP/HTTPS/TLS, SSL certificate basics

Week 3: SSH & Docker Foundations

Secure remote access and containerization basics — the same environment used throughout the Backend Developer track's Quality and Ops section.

Day 11 SSH & Key-Based Authentication

- The SSH client and server
- Generating and distributing SSH keys
- The SSH client configuration file
- Copying files with scp/rsync

Practical Task: Generate an SSH key pair, connect to a practice host, and transfer a file with scp

Source: Original course: SSH and key based authentication

Day 12 Containers vs Virtual Machines & Docker Install

- How containers differ from virtual machines
- Isolation and the shared kernel concept
- Installing Docker; engine and client

Practical Task: Install Docker and verify with a hello-world container

Source: Original course: Container/VM concepts, Docker architecture and installation

Day 13 Docker Images & Container Lifecycle

- Pulling and listing images from a registry
- Creating, running, stopping, inspecting containers
- Viewing logs and exec'ing into a running container

Practical Task: Pull an image, run it, inspect it, view logs, and exec into the running container

Source: Original course: Images and containers, Docker Hub and registries, Container lifecycle

Day 14 Volumes, Docker Networking & Compose

- Persisting data with volumes and bind mounts
- Bridge networks and container-to-container communication
- Docker Compose: defining and running multi-container apps

Practical Task: Write a docker-compose.yml that runs an app service linked to a database service

Source: Original course: Volumes and bind mounts, Networking in Docker, Docker Compose

Day 15 Building Images with Dockerfiles

- Dockerfile instructions (FROM, COPY, RUN, CMD)
- Layering and caching
- Basic best practices: least privilege, small base images

Practical Task: Write a Dockerfile for a simple app, build the image, and run a container from it

Source: Original course: Building images with Dockerfiles, Best practices

Week 4: Application Environment & Capstone

Logging, API testing, configuration, and a capstone that ties the month together into one working local development setup.

Day 16 Logs, Cron & Application Troubleshooting Basics

- Reading logs with journalctl and traditional log files
- Scheduling work with cron (crontab syntax)
- Basic troubleshooting workflow: reproduce, isolate, check logs

Practical Task: Schedule a script with cron and verify its output via logs

Source: Original course: Logs with journalctl, Cron and timers

Day 17 Working with APIs from the Command Line

- Manual API testing with curl and HTTPie
- Introduction to Postman collections
- JSON request and response handling patterns

Practical Task: Use curl/HTTPie/Postman to call a public REST API and inspect the response

Source: Original course: Diagnostic/HTTP concepts, bridges into Advanced course's API testing section

Day 18 Environment Configuration & Secrets

- Environment variables and shell configuration files
- Configuration and secrets via environment variables (not hardcoding)
- Virtual environments / dependency isolation concept

Practical Task: Configure an app to read settings from environment variables and a .env file

Source: Original course: Shell basics (env vars) / bridges into Advanced course's Quality and Ops section

Day 19 Storage Basics & Working with Files in Code

- Disks, partitions, and mounting (concept level)
- Checking disk and inode usage
- File systems from a developer's perspective: paths, permissions in deployed apps

Practical Task: Diagnose a simulated disk-space issue and resolve a file permission issue affecting an app

Source: Original course: Storage basics (condensed)

Day 20
Capstone:
Environment
Setup for a Real
Project

- Combine shell, Git, networking, SSH, and Docker skills
- Set up a complete local dev environment for a sample app
- Clone a repo, configure environment variables, build and run via Docker Compose

Practical Task: *Capstone: from a fresh machine, clone a sample project's Git repo, configure it, and get it running in containers — demo and walkthrough*

Source: *Capstone combining all prior days*

What Comes Next: Advanced Software Development Training

On completion of this Basic Training, developers move directly into the Advanced Software Development Training course, choosing (or being assigned to) one of three tracks:

- Frontend Developer — Web Foundations, Modern UI and Integration, Framework and Tooling, Quality and Delivery
- Backend Developer — Foundations, Data Layer, APIs and Framework, Quality and Ops
- UI and UX Designing — Foundations and Design Basics, Wireframing and Mobile UI Design, UX Thinking and Web Design, Real Projects and Portfolio Building

Each track in the Advanced course assumes the shell, Git, networking, SSH, and Docker fluency built in this Basic Training, allowing trainees to focus entirely on track-specific skills from day one.